

Data Balancing

CompTIA SecAI+: AI Security Certification Complete Exam Prep 2026 | Section 6: Data Security for AI

1. Why Class Imbalance Is Structural in Security AI

Security data is inherently imbalanced. On any given day, the overwhelming majority of network traffic is legitimate, the vast majority of files are benign, and most user actions are routine. Attacks, by definition, are exceptional events. This is desirable operationally — we want attacks to be rare — but it creates a structural problem for machine learning: the examples the model most needs to learn to detect are exactly the examples it sees least often during training.

The CompTIA SecAI+ exam tests whether candidates understand not just that imbalance exists, but why it degrades specific model behaviors, which metrics expose that degradation, and which techniques address it at each stage of the pipeline. This guide goes beyond the lecture to provide the depth needed for both the exam and operational practice.

2. How Imbalance Corrupts Model Learning

Most ML training algorithms minimize overall prediction error across the training set. With a 99:1 normal-to-attack ratio, a model that predicts 'normal' for every input achieves 99% accuracy while detecting zero attacks. This is not a bug in the algorithm — it is the algorithm behaving exactly as instructed, optimizing the objective it was given. The objective is wrong for the problem.

The mechanism: minority class examples (attacks) contribute so little to the total loss that the model receives almost no gradient signal from them. Mathematically, the model converges to a solution dominated entirely by the majority class. The result is a classifier that is trivially accurate but operationally worthless — a security tool that does not detect threats.

3. Metrics That Reveal vs. Hide Imbalance Problems

Metric	What It Measures	Problem with Imbalance	When to Use
Overall Accuracy	% of all predictions correct	Dominated by majority class; 99% accuracy possible with 0% attack detection	Never use as primary metric for imbalanced security data
Precision (per class)	Of predicted positives, what % are real positives	Shows whether alerts are meaningful; low precision = high false positive rate	Use for alert quality; combine with recall
Recall / Sensitivity (per class)	Of real positives, what % are detected	The most critical metric for attack detection — missing attacks is the core failure mode	Always report for minority (attack) class explicitly
F1 Score (per class)	Harmonic mean of precision and recall	Balances both; use macro-F1 to weight classes equally regardless	Use when both false positives and false negatives matter

		of size	
AUC-ROC	Area under the precision-recall tradeoff curve across all thresholds	Less sensitive to imbalance than accuracy; summarizes performance across operating points	Use for model comparison and threshold selection
Confusion Matrix	Per-class breakdown of TP, FP, FN, TN	Directly shows minority class miss rate; not masked by overall accuracy	Always include; makes minority class performance visible

Exam Rule: When evaluating security AI models on imbalanced data, always report recall for the minority (attack) class explicitly. A model that misses all attacks is unacceptable regardless of its overall accuracy.

4. Real-World Incident: Healthcare Fraud Detection Model Failure (2019)

Real-World Incident — Healthcare Algorithm Bias Study (Obermeyer et al., 2019): Researchers at UC Berkeley and Brigham and Women's Hospital published findings in Science showing that a widely deployed commercial healthcare risk-scoring algorithm systematically underestimated illness severity in Black patients compared to equally ill White patients. The root cause was class imbalance and a proxy metric problem: the algorithm used healthcare cost as a proxy for illness severity. Because Black patients historically had lower costs (due to systemic access barriers), the minority class — patients who were equally sick but lower-cost — was underweighted in training. While not a security AI system, the principle is directly applicable: models trained on imbalanced data with poorly chosen metrics optimize for the majority pattern, systematically failing on minority populations. In security, the 'minority population' is the attacker, and the failure mode is zero attack detection.

Key takeaway: metric choice and class balance are not just technical hygiene issues — they determine whether a model serves its actual purpose. In security AI, a model that achieves excellent aggregate performance while detecting no attacks is not a security tool; it is a false confidence generator.

5. Undersampling: Reducing the Majority Class

Undersampling reduces the number of majority class examples until class ratios are more balanced. The simplest approach — random undersampling — discards majority class examples uniformly at random. More sophisticated techniques retain majority class examples that are most informative (near decision boundaries) and remove redundant ones (deep within the majority region).

- **When to use:** Majority class data is genuinely abundant and some redundancy exists. Removing examples does not sacrifice unique information.
- **Risk:** Discarding potentially useful majority class examples that define the decision boundary between classes.

- **Advanced variant — Tomek Links:** Remove majority class examples that are nearest neighbors of minority class examples. This cleans the boundary region without removing the bulk of majority data.
- **Advanced variant — Edited Nearest Neighbors (ENN):** Remove majority class examples that are misclassified by their K nearest neighbors. This removes noisy majority examples near the minority class boundary.

6. Oversampling: Increasing the Minority Class

Oversampling duplicates minority class examples until the dataset is more balanced. Random oversampling copies existing minority examples exactly. While effective at balancing class counts, exact duplication increases overfitting risk: the model sees identical examples many times and may memorize them rather than learning generalizable features.

Combined with data augmentation (see previous lecture), oversampling becomes much more powerful: instead of duplicating exact copies, augmented oversampling generates varied versions of each minority example. This provides the quantity of simple oversampling with the diversity of augmentation. The combination is a standard baseline for security AI training pipelines.

7. SMOTE: Synthetic Minority Over-sampling Technique

SMOTE generates new synthetic minority class examples by interpolating between existing ones. The algorithm selects a minority class example, finds its K nearest minority class neighbors, and creates a new synthetic example at a random point along the line segment connecting the original to one of its neighbors.

SMOTE Variant	What It Adds	Best Used When
Standard SMOTE	Interpolates between any minority class neighbors	Minority class forms compact, reasonably separated clusters
Borderline-SMOTE	Interpolates only near the decision boundary where misclassification is most likely	Majority and minority classes overlap; boundary region needs reinforcement
ADASYN (Adaptive Synthetic Sampling)	Generates more synthetic examples in regions where the model performs worst (high misclassification density)	Minority class has complex, non-uniform distribution with multiple sub-regions of difficulty
SMOTENC	Handles datasets with a mix of numerical and categorical features	Security datasets with categorical fields (protocol type, OS family, attack category)

SMOTE operates in feature space, not raw data space. This means it is well-suited for tabular security features (packet statistics, behavioral metrics) but does not directly apply to raw bytes, images, or text without first extracting numeric feature representations.

8. Class Weights and Cost-Sensitive Learning

Class weighting adjusts the training loss function to penalize errors on minority class examples more heavily, without modifying the dataset at all. A weight of 10 on the minority class means that each minority class misclassification contributes 10 times more to the total loss than a majority class misclassification. The model is compelled to attend to minority class patterns because the cost of ignoring them is amplified.

Cost-sensitive learning is a generalization that specifies different costs for different types of errors. In security, the asymmetry is often extreme:

- Missing a ransomware attack (false negative): potential cost in millions of dollars of downtime, recovery, and regulatory fines.
- Generating a false alarm (false positive): cost of analyst time to investigate, typically minutes to hours.
- This asymmetry justifies setting false negative costs orders of magnitude higher than false positive costs, aligning model optimization directly with business impact rather than abstract accuracy metrics.

9. Ensemble Methods for Imbalanced Data

Ensemble methods combine predictions from multiple models, each trained on a different balanced view of the data. For imbalanced datasets:

- **BalancedBaggingClassifier:** Each bootstrap sample is balanced by undersampling the majority class to match the minority. Multiple such samples train multiple base classifiers, whose predictions are aggregated. No single classifier's imbalance bias dominates the ensemble.
- **Balanced Random Forest:** An extension of random forests that applies balanced bootstrap sampling at each tree, producing an ensemble robust to severe imbalance.
- **EasyEnsemble:** Constructs an ensemble from multiple undersampled majority-class subsets. Each subset is balanced; the ensemble covers the full majority class distribution across its members.
- **Why ensembles work for imbalance:** Different models see different balanced views of the data, preventing the pathological solution where the majority class always wins. Their combination produces predictions that reflect the full class structure.

10. Threshold Adjustment: Post-Training Tuning

Every probabilistic classifier outputs a confidence score that is thresholded to produce a binary decision. The default threshold (typically 0.5) assumes balanced classes. For imbalanced security data, lowering the threshold for the minority class increases recall (more attacks detected) at the cost of precision (more false positives).

The ROC curve plots true positive rate (recall) against false positive rate at every possible threshold. The precision-recall curve plots precision against recall. For imbalanced problems, the precision-recall curve is more informative. Threshold selection should be driven by the operational requirement:

- High-stakes environments (critical infrastructure, financial systems): prioritize recall, accept higher false positive rates. Missed attacks cost more than investigated false alarms.

- High-volume, analyst-constrained environments: balance precision and recall to avoid overwhelming analysts with false positives that reduce attention to real alerts.
- Regulatory compliance contexts: threshold may be mandated by the required detection rate (e.g., 'detect 95% of known ransomware variants').

11. Stratified Sampling for Train/Test Splits

When splitting an imbalanced dataset into training, validation, and test sets, random splitting risks producing test sets with too few minority class examples to evaluate detection performance reliably. Stratified sampling preserves the class ratio of the full dataset in each split. For a dataset with 1,000 attacks and 100,000 normal examples:

- A 70/15/15 split without stratification might leave only 80 attack examples in the test set by chance — insufficient for reliable evaluation.
- Stratified splitting guarantees ~150 attacks in the test set (15% of 1,000), enabling statistically meaningful evaluation of attack detection.
- Stratified K-fold cross-validation applies the same principle across all K folds, ensuring every fold contains proportional minority class representation.

12. Analogy: Security Imbalance as Emergency Room Triage

Memory Anchor — The Emergency Room Analogy: An emergency room sees hundreds of patients with minor complaints for every patient with a life-threatening condition. A triage nurse who processed patients in order of arrival — ignoring severity — would correctly handle 99% of patients but let critically ill patients die. Security AI faces the same problem: a model that handles all inputs with equal weight learns to serve the majority (normal traffic) and ignore the minority (attacks) that actually matter. Triage protocols are threshold adjustments and cost-sensitive learning — they explicitly override majority-class processing to prioritize critical minority cases.

13. Monitoring Imbalance Over Time in Production

Class distributions in production data change continuously. A new attack campaign can shift the attack/normal ratio. A security patching cycle can reduce one attack type's prevalence. Seasonal changes in user behavior alter the normal traffic baseline. A balancing strategy calibrated on training data from six months ago may be misaligned with today's threat landscape.

Production monitoring should track:

- Class ratio drift: alert when the ratio of detected minority class examples deviates significantly from the training distribution.
- Model performance on held-out labeled data: run periodic evaluations against labeled test sets to detect degradation in minority class recall.
- Alert volume and false positive rate trends: sustained increases in false positives or sustained decreases in confirmed detections signal distribution shift.

- Retraining triggers: define quantitative thresholds for performance degradation that automatically trigger retraining with freshly balanced data.

Key Terms Glossary

Term	Definition
Class Imbalance	A dataset condition where one or more classes have significantly fewer examples than others, causing models to underperform on minority classes
Undersampling	Reducing the number of majority class examples to create a more balanced training dataset
Oversampling	Duplicating or generating additional minority class examples to create a more balanced training dataset
SMOTE	Synthetic Minority Over-sampling Technique; generates synthetic minority class examples by interpolating in feature space between existing minority class samples
Class Weight	A scalar multiplier applied to the loss contribution of a class's examples during training, increasing the model's sensitivity to that class
Cost-Sensitive Learning	A training approach that assigns different costs to different types of misclassification errors, aligning model optimization with business impact
Threshold Adjustment	Post-training modification of the classification probability cutoff to shift the precision-recall tradeoff toward the desired operating point
Stratified Sampling	A data splitting technique that preserves class proportions in each resulting subset, ensuring minority classes are adequately represented in test and validation sets
Balanced Random Forest	An ensemble classifier that uses balanced bootstrap samples at each tree, making it robust to class imbalance
Precision-Recall Curve	A performance visualization that plots precision against recall at all possible thresholds; more informative than ROC for imbalanced datasets
Macro-F1 Score	The F1 score averaged across all classes with equal weight regardless of class size; penalizes poor performance on minority classes
False Negative Rate	The proportion of actual positive (attack) examples incorrectly classified as negative (normal); the primary failure metric for security detection systems

Quick Revision Checklist

- Security data is structurally imbalanced because attacks are rare — this is expected but creates ML training problems.

- Models trained on imbalanced data without correction learn to predict the majority class, achieving high accuracy while detecting no attacks.
- Overall accuracy is a misleading metric for imbalanced security data; always report per-class precision and recall, especially attack class recall.
- Undersampling reduces the majority class; oversampling duplicates or synthesizes minority class examples; both create a more balanced training set.
- SMOTE generates synthetic minority class examples by interpolating in feature space; variants (Borderline-SMOTE, ADASYN) target specific distribution challenges.
- Class weighting adjusts training loss without changing the dataset; cost-sensitive learning extends this by assigning asymmetric error costs reflecting real business impact.
- Ensemble methods like Balanced Random Forest train on multiple balanced subsets, preventing any single imbalanced view from dominating predictions.
- Threshold adjustment is a post-training step; lower thresholds increase minority class recall at the cost of precision.
- Stratified sampling must be used for train/test splits and cross-validation to ensure minority class representation in all dataset subsets.
- Class distributions drift in production — monitor class ratios, model performance, and alert volumes continuously and define retraining triggers.
- Combining techniques (undersample + oversample + class weights + threshold tuning) typically outperforms any single technique; experiment and validate on minority class metrics.

10 Exam-Based MCQs — Data Balancing

Test yourself after reading. These questions are written in real exam style — scenario-heavy, with plausible distractors. Read every explanation, including for questions you answered correctly.

Q1. Scenario: A financial institution's security team is retraining a network traffic classifier to detect a new ransomware campaign. The training dataset contains 500 labeled ransomware traffic captures and 250,000 normal traffic samples. The previous model achieved 98.7% overall accuracy but detected only 12% of ransomware events in production. Which metric should the team PRIORITIZE when evaluating the retrained model's performance?

- A) Overall accuracy across all traffic classes
- B) Recall for the ransomware traffic class specifically
- C) Precision for the normal traffic class
- D) F1 score for the majority class

Answer: B **Explanation:** B is correct because the team's operational goal is detecting ransomware, which is the minority class. Recall for the ransomware class measures the proportion of actual ransomware events that the model correctly identifies — missing them is the critical failure mode. A is wrong because overall accuracy is dominated by the majority (normal traffic) class and can be high even when the model detects no ransomware. C is wrong because precision for the normal class tells us how well the model avoids false positives on normal traffic, not whether it detects ransomware. D is wrong because majority class F1 is again dominated by the large normal traffic class and does not reflect ransomware detection capability.

Q2. Scenario: A security analyst is building a network intrusion detection model. The training dataset has 500 confirmed attack examples and 250,000 normal traffic examples. The normal traffic captures span many unique user behavior patterns, business applications, and network protocols that the team wants the model to learn. Which technique BEST addresses the imbalance without risking the loss of unique normal traffic patterns that define the decision boundary?

- A) Random undersampling of the normal traffic class to 5,000 examples
- B) SMOTE applied to the ransomware class to generate additional synthetic examples
- C) Discarding all normal traffic examples and retraining on only ransomware data
- D) Applying a uniform class weight of 0.01 to the ransomware class

Answer: B Explanation: B is correct because SMOTE generates new synthetic ransomware examples by interpolating between existing ones, increasing minority class representation without removing any normal traffic data. This avoids the risk of discarding informative normal examples that help define the classifier's decision boundary. A is wrong because random undersampling discards the majority of normal traffic examples, potentially removing unique patterns that help the model distinguish normal from attack traffic. C is wrong because removing all normal traffic examples eliminates the class entirely, making binary classification impossible. D is wrong because a class weight of 0.01 applied to the minority class would further reduce its influence, worsening the imbalance problem rather than addressing it.

Q3. Scenario: A cybersecurity firm's endpoint detection and response (EDR) product uses a machine learning model to classify process behaviors as normal or ransomware. The model was trained on an imbalanced dataset with 1,200 ransomware samples and 2 million normal behavior samples. Post-deployment monitoring shows the model detects 78% of ransomware events. The CISO requires that the endpoint detection model detect at least 95% of ransomware infections at an acceptable false positive rate. Which approach MOST directly enables the team to meet this requirement?

- A) Increase training dataset size by collecting more normal endpoint behavior data
- B) Switch from a decision tree classifier to a deep neural network to improve overall accuracy
- C) Adjust the classification probability threshold downward until ransomware recall reaches 95%, then evaluate the resulting false positive rate
- D) Apply stratified K-fold cross-validation during training to ensure balanced evaluation

Answer: C Explanation: C is correct because the requirement is expressed as a specific recall threshold (95% detection rate). Threshold adjustment directly controls the precision-recall tradeoff: lowering the threshold increases recall (more ransomware detected) at the cost of increased false positives. The team plots the precision-recall curve, identifies the threshold where recall reaches 95%, and evaluates whether the resulting false positive rate is operationally acceptable. A is wrong because adding more normal data worsens the imbalance and does not address the recall requirement for the minority class. B is wrong because switching model architectures does not inherently change how the model handles class imbalance; the same imbalance problem will affect a neural network if not explicitly addressed. D is wrong because stratified cross-validation is an evaluation technique, not a deployment technique — it helps measure performance but does not control the threshold during inference.

Q4. Which of the following BEST describes why overall accuracy is a misleading metric for evaluating security AI models trained on imbalanced data?

- A) Accuracy cannot be computed when the dataset has more than two classes
- B) A model that predicts the majority class for all inputs achieves high accuracy despite detecting no minority class examples
- C) Accuracy metrics are only valid when computed on test sets of at least 10,000 examples
- D) Security models require F1 scores rather than accuracy because F1 accounts for encryption overhead

Answer: B **Explanation:** B is correct because accuracy measures the overall proportion of correct predictions. With a 99:1 majority-to-minority ratio, a trivial classifier that always predicts 'normal' achieves 99% accuracy — seemingly excellent — while detecting zero attacks. This is the core reason accuracy is misleading for imbalanced security data. A is wrong because accuracy can be computed for any number of classes; multi-class accuracy is simply the proportion of correctly labeled examples across all classes. C is wrong because there is no minimum test set size requirement that makes accuracy a valid metric for imbalanced data; the problem is the metric itself, not the sample size. D is wrong because F1 is preferred for imbalanced data because it combines precision and recall, not because of anything related to encryption.

Q5. A security data scientist wants to use SMOTE to improve a malware classifier but the dataset contains both numerical features (file size, entropy, import count) and categorical features (file extension, packer type, compiler). Which SMOTE variant should be used?

- A) Borderline-SMOTE, which focuses on boundary region interpolation
- B) SMOTENC, which handles datasets with mixed numerical and categorical features
- C) ADASYN, which applies adaptive density-based sampling
- D) Standard SMOTE with one-hot encoding applied to all categorical features before interpolation is acceptable in all cases

Answer: B **Explanation:** B is correct because SMOTENC (SMOTE for Nominal and Continuous features) is specifically designed for mixed-type datasets. It applies standard SMOTE interpolation for numerical features while handling categorical features by randomly selecting values from the minority class neighbors, rather than interpolating between them (which is mathematically meaningless for categories). A is wrong because Borderline-SMOTE focuses on boundary region sampling but still requires numerical features; it does not handle categorical data. C is wrong because ADASYN addresses class distribution density but also does not natively handle categorical features. D is wrong because one-hot encoding categorical features before interpolation can produce synthetic samples with non-integer or impossible values for the encoded columns (e.g., 0.6 one-hot-encoded), which does not correspond to any valid category — SMOTENC is the correct tool.

Q6. What is the PRIMARY advantage of cost-sensitive learning over simple class weighting for security AI applications?

- A) Cost-sensitive learning automatically generates synthetic training examples to balance classes

- B) Cost-sensitive learning allows different error types (false positive vs. false negative) to be assigned different costs reflecting their actual business impact
- C) Cost-sensitive learning eliminates the need for threshold adjustment after training
- D) Cost-sensitive learning is computationally faster than class weighting on large datasets

Answer: B **Explanation:** B is correct because cost-sensitive learning assigns separate costs to each cell of the confusion matrix — e.g., a false negative (missed attack) costs \$500,000 in expected incident response; a false positive (false alarm) costs \$200 in analyst time. This aligns model optimization directly with business impact rather than statistical class balance. Class weighting typically uses a single multiplier per class applied symmetrically to all errors for that class. A is wrong because cost-sensitive learning modifies the loss function; it does not generate training data. C is wrong because threshold adjustment remains a valid tuning step even with cost-sensitive training; the optimal threshold depends on deployment context. D is wrong because cost-sensitive learning involves additional computation to apply asymmetric loss functions; it is not inherently faster.

Q7. A team trains an intrusion detection model, splits the data randomly 80/20 for training and test, and reports excellent attack detection in testing. However, production performance is poor. What is the MOST LIKELY explanation?

- A) The model was overfitted to the training set due to insufficient regularization
- B) Random splitting on an imbalanced dataset produced a test set with very few attack examples, making test performance unreliable
- C) The team should have used a 70/30 split instead of an 80/20 split
- D) Production network traffic has different encryption than training data

Answer: B **Explanation:** B is correct because with an imbalanced dataset (e.g., 1,000 attacks in 1,000,000 total samples), a random 80/20 split might place only 150-200 attack examples in the test set by chance. A model that detects 85% of those 150 examples appears to perform well, but 150 examples is insufficient to reliably estimate performance on a diverse attack population. The small, potentially non-representative test set produces optimistic but unreliable estimates. Stratified splitting would ensure the test set contains exactly 20% of attack examples (200 confirmed), with the right class ratio. A is wrong because overfitting would manifest as good training performance and poor test performance — but the scenario describes good test performance; production failure suggests the test set was unrepresentative. C is wrong because the split ratio is not the key issue; the missing step is stratification. D is wrong because encryption differences between environments would manifest as feature-level distribution shift, which is a separate issue from the evaluation methodology problem described.

Q8. Which combination of techniques is MOST likely to produce the best minority class recall for a severely imbalanced security dataset (1:10,000 attack-to-normal ratio)?

- A) Random undersampling alone, reducing the normal class to 1:1 ratio
- B) SMOTE combined with undersampling, class weighting, and threshold adjustment
- C) Oversampling by exact duplication of all attack examples 10,000 times
- D) Switching to an unsupervised anomaly detection approach to avoid the imbalance problem entirely

Answer: B **Explanation:** B is correct because a hybrid approach addresses imbalance from multiple angles simultaneously. SMOTE generates diverse synthetic attack examples (avoiding duplication overfitting). Undersampling reduces the extreme majority class size. Class weighting further compels the model to attend to attack examples during training. Threshold adjustment fine-tunes the final operating point for the desired recall level. The combination compensates for the limitations of each individual technique. A is wrong because random undersampling alone at 1:10,000 would discard 99.99% of normal examples, likely eliminating significant information about the majority class distribution. C is wrong because exact duplication 10,000 times creates extreme overfitting — the model memorizes attack examples rather than learning generalizable patterns. D is wrong because unsupervised approaches shift, not solve, the problem — anomaly detection requires its own calibration and still faces the challenge of distinguishing rare attacks from rare but legitimate behaviors.

Q9. A security ML model's attack detection rate drops from 91% to 74% over six months in production without any model changes. What is the MOST LIKELY cause?

- A) The model's weights have degraded due to accumulated memory errors in the inference server
- B) Class distribution drift — the proportion of attack patterns has shifted since training, misaligning the model's learned boundaries with current threat patterns
- C) The threshold adjustment applied at deployment has decayed over time
- D) SMOTE-generated examples in the training data have expired and are no longer effective

Answer: B **Explanation:** B is correct because security threat landscapes evolve continuously. New attack techniques, tool updates, and changes in attacker behavior shift the statistical distribution of attack examples in production data. A model trained on data from six months ago has decision boundaries calibrated to past attack patterns. When current attacks differ from those patterns, detection rate decreases — this is class distribution drift (also called concept drift or data drift). A is wrong because ML model weights are fixed values stored in memory; they do not degrade due to hardware errors in the way described. C is wrong because classification thresholds are static configuration values applied at inference time; they do not decay or change unless explicitly modified. D is wrong because SMOTE generates training examples; once training is complete, the synthetic examples have no further existence in the deployed model.

Q10. Which of the following BEST describes the purpose of Balanced Random Forest for imbalanced security datasets?

- A) It applies SMOTE to generate synthetic examples before each tree is trained in the ensemble
- B) It creates multiple balanced bootstrap subsets by undersampling the majority class, trains a decision tree on each, and aggregates predictions across all trees
- C) It assigns higher class weights to minority class examples in a single decision tree trained on the full imbalanced dataset
- D) It applies threshold adjustment to the output of a standard random forest to improve minority class recall

Answer: B **Explanation:** B is correct because Balanced Random Forest specifically uses balanced bootstrap sampling: for each tree in the ensemble, the majority class is undersampled to match the

minority class before sampling. Each tree sees a balanced view of the data, so no tree is dominated by the majority class. Aggregating across many such trees gives a final prediction that reflects the full majority class distribution while maintaining sensitivity to the minority class. A is wrong because Balanced Random Forest uses undersampling, not SMOTE, at the bootstrap stage; SMOTE is a separate technique that can be combined with other methods. C is wrong because class weighting modifies the loss function within a single model; Balanced Random Forest achieves balance through dataset resampling across an ensemble, which is fundamentally different. D is wrong because threshold adjustment is applied after training; it does not describe the training procedure of Balanced Random Forest.